NumPy Practice Questions

15 questions. The data for each question is given. Write your answers in your own notebook or in Colab.

Rules

- No `for` loops. Every task should take one or two lines.
- Some tasks show an **Expected output** — only the ones where mistakes are most likely.
- Where no expected output is given, judge for yourself whether your answer looks correct.

GIVEN DATA

```
import numpy as np
print(np.__version__)
```

Q1. Building the class register

Three students' marks arrived as three separate lists, and one old record arrived as text.

Tasks

1. Build one table from `s1`, `s2` and `s3`.
2. From that table, print the number of students and the number of subjects as two separate numbers.
3. Print the total count of marks entries stored in the table.
4. Store the same table as decimals instead of whole numbers.
5. Convert `old` into a form where arithmetic works, then print the total marks of both students.
6. Attach `old` below the main table so that all five students sit in one table.

Expected output — task 5

```
[268 246]
```

Expected output — task 6

```
[[ 72  85  60  91]
 [ 45  30  98  55]
 [ 88  66  74 100]
 [ 62  71  55  80]
 [ 49  58  66  73]]
```

GIVEN DATA

```
s1 = [72, 85, 60, 91]
s2 = [45, 30, 98, 55]
s3 = [88, 66, 74, 100]

old = np.array([['62', '71', '55', '80'],
                ['49', '58', '66', '73']])
```

Q2. Templates and sequences

Tasks

1. Create a blank mark sheet for 6 students and 4 subjects, every entry starting at 0.
2. Create a table of the same size where every student is temporarily given 35.
3. A batch has roll numbers running from 101 to 130. Generate only the alternate roll numbers, and print how many were generated.
4. Split a 3-hour practical into 7 equally spaced time markers, including both the start and the end.
5. Create a 5 × 5 table that has 10 on the diagonal and 0 everywhere else.
6. Create a 4 × 4 table that has 100 on the diagonal and 35 everywhere else.

Expected output — task 3

```
[101 103 105 107 109 111 113 115 117 119 121 123 125 127 129]
15
```

Expected output — task 4

```
[0.  0.5 1.  1.5 2.  2.5 3. ]
```

Expected output — task 6

```
[[100  35  35  35]
 [ 35 100  35  35]
 [ 35  35 100  35]
 [ 35  35  35 100]]
```

Q3. The attendance row

Tasks

1. Print the first and the last student’s attendance without writing the class strength anywhere in your code.
2. Print the attendance of every alternate student, starting from the second one.
3. Print the last four values.
4. Reverse the row, then print every third value of the reversed row.
5. Change the attendance of the two middle students to 30 each.
6. Compare the first half of the row with the second half position by position, and print the difference.
7. Print the top three values of the row after reversing it.

Expected output — task 4

```
[23  9 12 22]
```

Expected output — task 6

```
[-2  9  4 -5 -3]
```

GIVEN DATA

```
present = np.array([22, 18, 25, 12, 20, 24, 9, 21, 17, 23])
```

Q4. Reading the mark sheet

The columns are Math, Physics, Chemistry and English.

Tasks

1. Print the Physics and Chemistry marks of the middle three students only.
2. Print the last subject’s marks for the whole class.
3. Produce a table containing only the first and the last subject of every student.
4. Print the mark sheet with the students in reverse order.
5. Increase only the Chemistry marks of the whole class by 5, leaving the rest unchanged.
6. Print the mark that sits in the last row and the second-last column.
7. Produce a table of the last two students and their first two subjects.

Expected output — task 1

```
[[30 98]
 [66 74]
 [92 55]]
```

Expected output — task 3

```
[[ 72  91]
 [ 45  55]
 [ 88 100]
 [ 95  40]
 [ 30  70]]
```

GIVEN DATA

```
marks = np.array([[72, 85, 60, 91],
                  [45, 30, 98, 55],
                  [88, 66, 74, 100],
                  [95, 92, 55, 40],
                  [30, 48, 62, 70]])
```

Q5. Scholarship shortlisting

Tasks

1. Print the totals of students who scored above 250.
2. Print the totals of students scoring above 180 but below 250.
3. Print the totals of students who scored either below 150 or above 380.
4. Print how many students fall in the band of task 2.
5. Print the average of only those students who scored above 200.
6. Print the index position of every student below 150.
7. Print the highest total among the students who did not make the merit band of task 1.

Expected output — task 2

```
[245 199]
2
```

Expected output — task 6

```
[4 9]
```

GIVEN DATA

```
totals = np.array([245, 310, 178, 392, 140, 265, 199, 155, 288, 132])
```

Q6. After revaluation

Tasks

1. Increase every student’s marks by 15% and round to 2 decimals.
2. After the increase, no student may show more than 100. Produce the corrected list.
3. Print how many students gained less than 10 marks from this increase.
4. The pass mark is 40. Print how many students were failing before the increase and how many after.
5. Print each student’s marks as a percentage of the topper’s marks, rounded to 1 decimal.
6. Print the total number of marks the school gave away in this revaluation.

Expected output — task 2

```
[ 46.    63.25  71.3  100.   100.    37.95  81.65  66.7 ]
```

Expected output — task 5

```
[ 42.1  57.9  65.3  92.6 100.   34.7  74.7  61.1]
```

GIVEN DATA

```
marks = np.array([40, 55, 62, 88, 95, 33, 71, 58])
```

Q7. The result summary

Tasks

1. Print the class average and the middle value of the class.
2. Print the gap between the highest and the lowest marks.
3. Print the index positions of the topper and of the weakest student.
4. Print the second highest marks in the class.

5. Print how many students scored above the class average.
6. Print how many students lie within one standard deviation of the class average.
7. Print the class average after removing the topper.

Expected output — task 4

```
91
```

Expected output — task 6

```
5
```

Expected output — task 7

```
62.67
```

GIVEN DATA

```
marks = np.array([72, 45, 88, 95, 30, 67, 91, 54, 78, 39])
```

Q8. Subject-wise and student-wise reports

Tasks

1. Print the average of each subject and the total of each student.
2. Print the index of the toughest subject for the class.
3. Print the index of the student who topped overall.
4. For every subject, print the marks of the student who topped that subject.
5. Print how many students scored above 60 in each subject.
6. Print the index of the subject in which the class marks are the most spread out.
7. Print the total marks of the student who scored the lowest in Chemistry.

Expected output — task 4

```
[ 95  92  98 100]
```

Expected output — task 5

```
[3 3 3 3]
```

Expected output — task 6

```
0
```

GIVEN DATA

```
marks = np.array([[72, 85, 60, 91],
                  [45, 30, 98, 55],
                  [88, 66, 74, 100],
                  [95, 92, 55, 40],
                  [30, 48, 62, 70]])
```

Q9. The annual function draw

Tasks

1. Make the whole draw repeatable, then pick 3 winners from `participants`.
2. Generate the attendance percentages of these 10 participants as whole numbers between 60 and 100.
3. Generate a 4 × 3 table of random decimals between 0 and 1, then turn it into whole-number marks out of 100.
4. Shuffle the participant list and print the first four as the front-row seating.
5. Simulate 20 dice rolls and print how many times a 6 appeared.
6. From the attendance of task 2, print the roll numbers of the participants who crossed 80.

Note: the outputs below appear only if the seed `42` is set before each task. A different seed or a different call order will give different numbers, and that is not wrong.

Expected output — task 1

```
[107 104 108]
```

Expected output — task 2

```
[98 88 74 67 80 98 78 82 70 70]
```

Expected output — task 5

```
3
```

GIVEN DATA

```
participants = np.array([101, 102, 103, 104, 105, 106, 107, 108, 109, 110])
```

Q10. Grace marks and weightage

Tasks

1. Apply the subject-wise grace marks to the whole class, keeping every value at or below 100.
2. Multiply each subject by its own weightage and print the table rounded to 1 decimal.
3. Produce a table showing how far each mark sits from that subject's own class average.
4. Give 5 bonus marks to the first student, 0 to the second, 2 to the third and 0 to the fourth — in a single operation.
5. From the table of task 3, print how many entries lie below the subject average.
6. Produce a table where every subject is brought to one scale by subtracting its average and dividing by its spread. Round to 2 decimals.

Expected output — task 3

```
[[ -3.    16.75 -11.75  19.5 ]
 [-30.   -38.25  26.25 -16.5 ]
 [ 13.    -2.25   2.25  28.5 ]
 [ 20.    23.75 -16.75 -31.5 ]]
```

Expected output — task 4

```
[[ 77  90  65  96]
 [ 45  30  98  55]
 [ 90  68  76 102]
 [ 95  92  55  40]]
```

Expected output — task 6

```
[[-0.16  0.7  -0.7   0.79]
 [-1.56 -1.59  1.57 -0.67]
 [ 0.68 -0.09  0.13  1.15]
 [ 1.04  0.99 -1.   -1.27]]
```

GIVEN DATA

```
marks = np.array([[72, 85, 60, 91],
                  [45, 30, 98, 55],
                  [88, 66, 74, 100],
                  [95, 92, 55, 40]])

grace     = np.array([5, 0, 8, 3])
weightage = np.array([1.2, 1.0, 1.1, 1.0])
```

Q11. Distances and arrangements

Tasks

1. Build a 5 × 5 table of walking distances between every pair of hostel blocks, with all values positive.
2. For each block, print the distance to the block farthest from it.
3. Print the index of the block nearest to block 0, leaving out block 0 itself.
4. Print the two blocks that are closest to each other anywhere on the road.
5. Arrange roll numbers 1 to 24 into rows of 6 without typing the number of rows.

6. Turn `sanskrit` into a form that can be attached as a column to a mark sheet, using two different methods.
7. Turn `sanskrit` into a single-row table of shape `(1, 5)`.

Expected output — task 1

```
[[  0 300 700 100 500]
 [300   0 400 400 200]
 [700 400   0 800 200]
 [100 400 800   0 600]
 [500 200 200 600   0]]
```

Expected output — task 2

```
[700 400 800 800 600]
```

Expected output — task 7

```
[[65 70 88 92 54]]
```

GIVEN DATA

```
metres   = np.array([200, 500, 900, 100, 700])
sanskrit = np.array([65, 70, 88, 92, 54])
```

Q12. The master mark sheet

Tasks

1. Produce a flat one-line version of the master that can be edited without affecting the master. Change its first value to 999 and print the master.
2. Produce a flat version that shares memory with the master. Change its last value to 111 and print the master.
3. Set the entire middle row to 0 while leaving the master unchanged.
4. Double the first two columns without changing the master.
5. Double the first two columns, this time changing the master itself.
6. Produce a copy of the master in which every value above 50 is reduced by 10, leaving the master untouched.

Expected output — task 1 (master unchanged)

```
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

Expected output — task 2 (master changed)

```
[[ 10  20  30]
 [ 40  50  60]
 [ 70  80 111]]
```

Expected output — task 5

```
[[ 20  40  30]
 [ 80 100  60]
 [140 160  90]]
```

GIVEN DATA

```
master = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])
```

Q13. Merging section records

Tasks

1. Combine Section A and Section B into one mark sheet and print its shape.
2. Attach a totals column to that combined mark sheet.
3. Build a two-column table from `roll_C` and `marks_C`, roll numbers first.
4. Add a new student — roll 205, marks 73 — as a fresh row to the table of task 3.

5. Attach a serial-number column, starting from 1, to the left of the combined mark sheet of task 1.
6. From the combined sheet, produce a new sheet holding only the students whose total crosses 200.
7. Attach a grade column to the combined sheet: 225 and above → A, 150 to 224 → B, below 150 → C.

Expected output — task 2

```
[[ 72  85  60 217]
 [ 45  30  98 173]
 [ 88  66  74 228]
 [ 95  92  55 242]
 [ 30  48  62 140]]
```

Expected output — task 5

```
[[ 1 72 85 60]
 [ 2 45 30 98]
 [ 3 88 66 74]
 [ 4 95 92 55]
 [ 5 30 48 62]]
```

Expected output — task 7

```
[['72' '85' '60' 'B']
 ['45' '30' '98' 'B']
 ['88' '66' '74' 'A']
 ['95' '92' '55' 'A']
 ['30' '48' '62' 'C']]
```

GIVEN DATA

```
sec_A = np.array([[72, 85, 60],
                  [45, 30, 98],
                  [88, 66, 74]])

sec_B = np.array([[95, 92, 55],
                  [30, 48, 62]])

roll_C  = np.array([201, 202, 203, 204])
marks_C = np.array([67, 82, 45, 91])
```

Q14. Cleaning and grading

Tasks

1. Repair the attendance row: every value above 100 becomes 100, and every `-1` becomes the average of the genuine entries.
2. Label each repaired value: 85 and above → Regular, 60 to 84 → Average, below 60 → Short.
3. Print the index positions of the Short students along with their attendance values.
4. In the marks table, find the row and column position of every mark below 50.
5. Print the actual marks sitting at those positions.
6. Replace every mark below 35 with 35, and print how many entries were changed.
7. Print the index of the student who has the highest number of marks below 60.

Expected output — task 1

```
[ 88.    74.75 100.    76.    45.    74.75  92.   100.    67.    30.  ]
```

Expected output — task 4

```
rows: [1 3 3]
cols: [2 0 2]
```

Expected output — task 5

```
[32 40 45]
```

Expected output — task 7

```
3
```

GIVEN DATA

```
marks = np.array([[70, 65, 80],
                  [55, 90, 32],
                  [88, 76, 91],
                  [40, 52, 45],
                  [60, 58, 66]])

attendance = np.array([88, -1, 105, 76, 45, -1, 92, 120, 67, 30])
```

Q15. The final weighted result

Tasks

1. Compute the weighted final score of every student using `w`.
2. Print the index of the student with the highest weighted score.
3. Using a multiplication rather than `sum`, produce the subject-wise total marks of the whole class.
4. The board has proposed two weightage schemes, held in `W`. Compute both final scores of all four students in one multiplication.
5. From that result, print how many students score higher under the second scheme.
6. Print the difference between the two schemes for every student, rounded to 2 decimals.
7. Attach the better of the two scores as a fourth column to `M`.

Expected output — task 3

```
[273. 289. 309.]
```

Expected output — task 4

```
[[72.5 72. ]
 [72.3 67.1]
 [85.6 86.5]
 [61.8 61.4]]
```

Expected output — task 6

```
[-0.5 -5.2  0.9 -0.4]
```

GIVEN DATA

```
M = np.array([[70, 65, 80],
              [55, 90, 72],
              [88, 76, 91],
              [60, 58, 66]])

w = np.array([0.3, 0.3, 0.4])

W = np.array([[0.3, 0.5],
              [0.3, 0.2],
              [0.4, 0.3]])
```

Concept coverage

Q	Concepts
1	`np.array`, `shape` `size`, `dtype`, `astype`, `vstack`
2	`zeros` `ones` `full` `eye`, tuple shape, `arange`, `linspace`, `np.where`
3	1D indexing, negative indexing, step slicing, reversing, slice assignment
4	2D indexing, `[rows, columns]`, column slicing, step slicing, sub-tables
5	Boolean filtering, `&` and `|`, counting, filtered statistics, index positions
6	Element-wise arithmetic, percentage change, capping, comparison counting
7	`mean` `median` `std`, `min` `max`, `argmin` `argmax`, `.copy()`, filtered averages
8	`axis=0` vs `axis=1`, `argmax` per axis, per-column counting, spread comparison
9	`seed`, `randint`, `rand`, `choice`, `shuffle`, filtering random output
10	Row broadcasting, column broadcasting, column mean and std, same-scale conversion
11	Column-meets-row grid, `np.where`, `reshape(-1, n)`, `reshape(-1, 1)`, `np.newaxis`
12	`flatten` vs `ravel`, views vs copies, `.copy()`, in-place modification
13	`vstack` `hstack` `concatenate`, computed column, boolean row filtering, nested `np.where`
14	`np.where` cleaning, nested `np.where`, condition-only `np.where`, 2D coordinates
15	`@` vs `*`, matrix multiplication, transpose, multi-column weightage, `hstack`

Core seven: Q4, Q8, Q10, Q11, Q13, Q14, Q15 — clear these and the rest will follow.